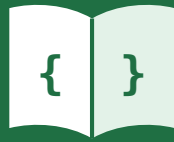

Modül 08 — Redis ve Önbellekleme

Ders Notları

Spring Boot Edu



2026-09-23

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Neden Önbellek?	2
2.2 Önbellekleme Desenleri	2
2.3 Önbellek Soyutlaması	3
3. Adım Adım Örnekler	3
3.1 @Cacheable: Önbellek Üzerinden Okuma	4
3.2 Redis Önbelleğini Yapılandırmak	5
3.3 Önbelleği Tutarlı Tutmak: @CachePut ve @CacheEvict	6
3.4 Veri Yapıları: Sorted Set ve List	7
3.5 Süresi Dolan Atomik Sayaçlar	8
3.6 Publish/Subscribe	9
3.7 Redis ile Spring Session	10
3.8 Testcontainers ile Test	11
4. Sık Yapılan Hatalar ve En İyi Pratikler	11
5. Özet	12
6. İleri Okuma	12

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Önbelleğin ne zaman işe yaradığını ve hangi önbellekleme desenini seçeceğinizi (cache-aside, write-through) açıklamak
- Metot sonuçlarını @Cacheable, @CachePut ve @CacheEvict ile önbelleğe almak
- Redis'i önbellek deposu olarak yapılandırmak: TTL, JSON değerler, null kaydı yok
- StringRedisTemplate ile Redis veri yapılarını kullanmak: sorted set, list ve atomik sayaçlar
- Redis publish/subscribe ile uygulamalar arasında mesaj göndermek
- Spring Session ile HTTP oturumunu Redis'te saklamak
- Redis kodunu @DataRedisTest ve Testcontainers ile test etmek

Ön koşullar: Modül 01-03 (bean'ler, yapılandırma, REST) · **Tahmini süre:** 4 saat · **Docker gerekir**

2. Kavramlar

2.1 Neden Önbellek?

Önbellek (cache), pahalı verinin bir kopyasını uygulamaya yakın tutar. Sonraki istek yavaş kaynağa (veritabanı, uzak servis) gitmek yerine bu kopyayı okur.

Uygun	Uygun değil
Sık okunur, seyrek değişir (ürün detayı, ayarlar)	Her istekte değişir (kampanya sırasında stok)
Hesaplaması veya getirmesi pahalıdır	Okuması zaten ucuzdur
Biraz eski veri kabul edilebilir	Her zaman kesin olmalıdır (hesap bakiyesi)

Redis, bellekte çalışan bir anahtar-değer deposudur. Tüm uygulama örnekleri aynı Redis'e bağlanır, böylece hepsi tek bir önbelleği paylaşır. Uygulama içi bir önbellek (ör. Caffeine) ise her örnekte ayrı yaşar.

2.2 Önbellekleme Desenleri

Desen	Okuma	Yazma	Bu modülde
Cache-aside	Önbelleğe bak. Yoksa kaynaktan oku ve sonucu sakla	Kaynağa yaz, önbellek kaydını sil	@Cacheable + @CacheEvict
Write-through	Cache-aside ile aynı	Kaynağa ve önbelleğe birlikte yaz	@CachePut

i Not

Her önbellek kaydının bir sonu olmalıdır. TTL (time to live, yaşam süresi) yoksa hiç silinmeyen bir kayıt sonsuza kadar eski kalır. Yazmalarda kaydı silseniz bile her önbelleğe bir TTL verin.

2.3 Önbellek Soyutlaması

Spring'in önbellek anotasyonları Redis'e bağlı değildir. @EnableCaching, bean'lerinizi bir proxy ile sarar. Proxy bir CacheManager kullanır. Classpath'te Redis varsa Spring Boot bir RedisCacheManager oluşturur. Aynı anotasyonlar Caffeine, Hazelcast (modül 09) veya basit bir map ile de çalışır.

! Önemli

Önbellekleme, @Transactional gibi proxy üzerinden çalışır. **Aynı sınıf içinde** bir metottan diğerine yapılan çağrı (self-call) proxy'yi, dolayısıyla önbelleği atlar.

3. Adım Adım Örnekler

Docker açıkken uygulamayı başlatın. Redis, kök dizindeki compose.yaml dosyasından (redis profili) otomatik başlar:

```
./mvnw -pl modules/08-redis-caching/lesson -am spring-boot:run
```

Tur (LessonTour), aşağıdaki örneklerin hepsini bir kez çalıştırır ve sonuçları yazdırır.

! Uyarı

Redis bilgisayarınızda ayrıca kuruluysa (ör. Homebrew ile), 127.0.0.1:6379 portunu zaten dinliyor olabilir. Uygulama bu durumda container'a değil o Redis'e bağlanır ve docker compose exec redis redis-cli hiç anahtar göstermez. lsof -iTCP:6379 -

```
sTCP:LISTEN ile kontrol edin ve yereldekini durdurun (brew services stop redis).
```

Ayarlar (spring: altında):

```
cache:
  redis:
    time-to-live: 1h                # default TTL for caches without their own
    ↪ configuration
  session:
    timeout: 30m                   # Spring Session: idle sessions expire in Redis too
```

3.1 @Cacheable: Önbellek Üzerinden Okuma

SlowBookRepository yavaş bir kaynak gibi davranır: her okuma 300 ms sürer ve sayılır. BookService, önbellek anotasyonlarını onun önüne koyar:

```
@Service
public class BookService {

    private final SlowBookRepository repository;

    public BookService(SlowBookRepository repository) {
        this.repository = repository;
    }

    @Cacheable(cacheNames = "books", sync = true)                // key = isbn;
    ↪ sync: one loader per key
    public Book find(String isbn) {
        Book book = repository.findByIsbn(isbn);
        if (book == null) {
            throw new BookNotFoundException(isbn);                // exceptions are
            ↪ not cached
        }
        return book;
    }

    @Cacheable(cacheNames = "books", unless = "#result == null") // do not cache
    ↪ "not found"
    public @Nullable Book findOrNull(String isbn) {
        return repository.findByIsbn(isbn);
    }

    @CachePut(cacheNames = "books", key = "#isbn")                // write-through:
    ↪ update source AND cache
    public Book changePrice(String isbn, BigDecimal newPrice) {
```

```

        Book updated = new Book(isbn, find(isbn).title(), newPrice); // find():
        ↪ self-call, not cached here
        repository.save(updated);
        return updated;
    }

    @CacheEvict(cacheNames = "books") // remove the stale
    ↪ entry
    public void remove(String isbn) {
        repository.delete(isbn);
    }
}

```

- `@Cacheable("books")`: İlk çağrı metodu çalıştırır ve sonucu `books::<isbn>` anahtarıyla saklar. Sonraki çağrılar saklanan değeri **metodu çalıştırmadan** döndürür.
- Varsayılan anahtar, metodun parametresidir. Birden çok parametre varsa anahtar açıkça seçin: `key = "#isbn"`.
- `sync = true`: Aynı anahtar aynı anda birçok thread bulamazsa yalnızca biri yükler, diğerleri sonucu bekler. Bu, yavaş kaynağa bir “cache stampede” (hücum) yapılmasını önler.
- Exception’lar önbelleğe alınmaz. `unless = "#result == null"` de null sonuçları atlar, yani “bulunamadı” saklanmaz.

Tur çıktısında ikinci okuma çok daha hızlıdır:

```

== 3.1 @Cacheable
1st find: Spring in Action in 322 ms
2nd find: Spring in Action in 1 ms

```

3.2 Redis Önbelleğini Yapılandırmak

Redis varsayılan olarak değerleri Java serileştirmesiyle saklar. Bu okunamaz ve sınıf değişince bozulur. Aşağıdaki yapılandırma, books önbelleğinin bunun yerine JSON saklamasını ve kendi TTL’ine sahip olmasını sağlar:

```

@Configuration(proxyBeanMethods = false)
@EnableCaching
public class CacheConfiguration {

    @Bean
    RedisCacheManagerBuilderCustomizer booksCache(RedisConnectionFactory
    ↪ connectionFactory) {
        return builder -> builder
            .cacheWriter(RedisCacheWriter.create(connectionFactory,

```

```

        writer -> writer.immediateWrites()) //
↪ Lettuce writes are async by default
        .withCacheConfiguration("books",
        ↪ RedisCacheConfiguration.defaultCacheConfig()
        .entryTtl(Duration.ofMinutes(10)) // stale
        ↪ data expires on its own
        .serializeValuesWith(SerializationPair.fromSerializer(
            new JacksonJsonRedisSerializer<>(Book.class))) //
        ↪ readable JSON, typed to Book
        .disableCachingNullValues());
    }
}

```

- RedisCacheManagerBuilderCustomizer, Boot'un oluşturduğu cache manager'ı değiştirir. Onu yerinden etmezsiniz.
- spring.cache.redis.time-to-live tüm önbellekler için varsayılandır. withCacheConfiguration tek bir önbelleğin değerlerini belirler.
- JacksonJsonRedisSerializer<>(Book.class) okunabilir JSON yazar ve onu Book olarak geri okur.
- disableCachingNullValues() null değerleri reddeder. unless ile birlikte önbellek hiçbir zaman "bulunamadı" saklamaz.

! Uyarı

Spring Data Redis 4, Lettuce ile önbellek kayıtlarını varsayılan olarak **asenkron** yazar: put ve evict, Redis yazmayı onaylamadan geri döner. Yazmadan hemen sonraki bir okuma hâlâ eski değeri görebilir. immediateWrites() her yazmanın Redis'i beklemesini sağlar. Spring Boot'ta bunun bir property'si yoktur, bu yüzden customizer ayarlar.

Kaydı redis-cli ile inceleyin:

```

docker compose exec redis redis-cli GET "books:9781617297571"
# {"isbn":"9781617297571","title":"Spring in Action","price":95.00}
docker compose exec redis redis-cli TTL "books:9781617297571"
# (integer) 598

```

3.3 Önbelleği Tutarlı Tutmak: @CachePut ve @CacheEvict

Bölüm 3.1'deki BookService listesinde changePrice ve remove metotlarına bakın:

- @CachePut metodu **her zaman** çalıştırır ve dönüş değerini saklar. Bu write-through'dur: Kaynak ve önbellek yeni fiyatı aynı çağrıda alır.
- @CacheEvict kaydı siler. Sonraki find yeniden kaynaktan okur (cache-aside).
- @CacheEvict(allEntries = true) tüm önbelleği boşaltır. Nadiren kullanın.

★ İpucu

Emin değilseniz `@CacheEvict` seçin. Silmek her zaman güvenlidir. `@CachePut` ile metodun dönüş değeri, `@Cacheable`'ın döndüreceği değerle birebir aynı olmalıdır. Aksi hâlde önbellekte kaynaktakinden farklı bir nesne durur.

3.4 Veri Yapıları: Sorted Set ve List

Redis bir önbellekten fazlasıdır. `StringRedisTemplate`, veri yapılarına erişim sağlar. **Sorted set**, üyeleri bir puana göre sıralı tutar ve puanı atomik olarak günceller:

```
@Component
public class BestSellers {

    private static final String KEY = "bestsellers";

    public record Entry(String title, long sold) {
    }

    private final StringRedisTemplate redis;

    public BestSellers(StringRedisTemplate redis) {
        this.redis = redis;
    }

    public void recordSale(String title, int quantity) {
        redis.opsForZSet().incrementScore(KEY, title, quantity); // ZINCRBY
    }

    public List<Entry> top(int count) {
        var ranked = redis.opsForZSet().reverseRangeWithScores(KEY, 0, count - 1); //
        ↪ ZREVRANGE ... WITHSCORES
        return Objects.requireNonNull(ranked).stream()
            .map(tuple -> new Entry(Objects.requireNonNull(tuple.getValue()),
                Math.round(Objects.requireNonNull(tuple.getScore()))))
            .toList();
    }
}
```

`LPUSH` + `LTRIM` ile bir **list**, sınırlı bir geçmiş tutar. Burada bir kullanıcının baktığı son beş kitap:

```
@Component
public class RecentlyViewed {
```

```

private final StringRedisTemplate redis;

public RecentlyViewed(StringRedisTemplate redis) {
    this.redis = redis;
}

public void view(String user, String isbn) {
    String key = "viewed:" + user;
    redis.opsForList().leftPush(key, isbn);           // LPUSH
    redis.opsForList().trim(key, 0, 4);               // LTRIM: keep
    ↪ the newest 5
}

public List<String> of(String user) {
    return Objects.requireNonNull(redis.opsForList().range("viewed:" + user, 0, -1));
}
}

```

Yapı	Tipik kullanım	Komutlar
String	Önbellek değerleri, sayaçlar	GET, SET, INCR, EXPIRE
List	Son öğeler, basit kuyruklar	LPUSH, LRANGE, LTRIM
Set	Tekil üyeler (etiketler, çevrimiçi kullanıcılar)	SADD, SISMEMBER
Sorted set	Sıralamalar, liderlik tabloları	ZINCRBY, ZREVRANGE, ZREVRANK
Hash	Alanları olan nesneler	HSET, HGETALL

3.5 Süresi Dolan Atomik Sayaçlar

INCR atomiktir: Bin eşzamanlı çağrı tam olarak bin sonucunu verir. “Oku, bir ekle, yaz” yarışı yoktur. Her gün için bir anahtar ve EXPIRE ile eski sayaçları Redis kendisi temizler:

```

@Component
public class PageViews {

    private final StringRedisTemplate redis;

    public PageViews(StringRedisTemplate redis) {
        this.redis = redis;
    }
}

```

```

public long count(String page) {
    String key = keyForToday(page);
    Long views = redis.opsForValue().increment(key);           // INCR: safe
    ↪ with many concurrent callers
    redis.expire(key, Duration.ofDays(2));                     // Redis deletes
    ↪ old counters itself
    return views == null ? 0 : views;
}

```

3.6 Publish/Subscribe

Pub/sub ile bir uygulama bir **kanala** mesaj gönderir. Redis bunu o anda abone olan tüm uygulamalara iletir. Yayıncı:

```

@Component
public class PriceChangePublisher {

    public static final String CHANNEL = "price-changes";

    private final StringRedisTemplate redis;
    private final ObjectMapper json;

    public PriceChangePublisher(StringRedisTemplate redis, ObjectMapper json) {
        this.redis = redis;
        this.json = json;
    }

    public void publish(PriceChange change) {
        redis.convertAndSend(CHANNEL, json.writeValueAsString(change)); // PUBLISH
    ↪ price-changes {...}
    }
}

```

Abone. RedisMessageListenerContainer bir bağlantıyı açık tutar ve her mesaj için listener'ı çağırır:

```

@Configuration(proxyBeanMethods = false)
public class PubSubConfiguration {

    @Bean
    RedisMessageListenerContainer priceChangeSubscription(RedisConnectionFactory
    ↪ connections,
                                                            PriceChangeListener listener) {
        var container = new RedisMessageListenerContainer();
        container.setConnectionFactory(connections);
    }
}

```

```

        container.addMessageListener(listener, new
↪ ChannelTopic(PriceChangePublisher.CHANNEL)); // SUBSCRIBE
        return container;
    }
}

```

!! Dikkat

Redis pub/sub “gönder ve unut” çalışır. Çevrimdışı bir abone mesajı kaçırmaz ve hiçbir şey saklanmaz. Kaybolmaması gereken olaylar için Redis Streams veya Kafka (modül 11) kullanın.

3.7 Redis ile Spring Session

Bir load balancer arkasında birden çok uygulama örneği varsa, bir örneğin belleğinde tutulan oturum, sonraki istek başka bir örneğe gittiğinde kaybolur. Spring Session, HttpSession’ı Redis’te saklar. Kod sıradan servlet kodu olarak kalır:

```

@RestController
public class VisitController {

    @GetMapping("/api/visits")
    public String visit(HttpSession session) {
        Integer visits = (Integer) session.getAttribute("visits");
        int count = visits == null ? 1 : visits + 1;
        session.setAttribute("visits", count); // written to Redis at
↪ the end of the request
        return "visits in this session: " + count;
    }
}

```

spring-boot-starter-session-data-redis bağımlılığı yeterlidir. Gerisini Boot yapılandırır. spring.session.timeout boşta kalma süresini belirler. Deneyin:

```

curl -c /tmp/c -b /tmp/c localhost:8080/api/visits # visits in this session: 1
curl -c /tmp/c -b /tmp/c localhost:8080/api/visits # visits in this session: 2
docker compose exec redis redis-cli KEYS "spring:session:*"

```

Uygulamayı yeniden başlatıp tekrar çağırın: Sayaç kaldığı yerden devam eder, çünkü oturum Redis’te yaşar.

3.8 Testcontainers ile Test

```
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final RedisContainer REDIS = new
        ↪ RedisContainer(DockerImageName.parse("redis:8.8.3-alpine"));

    @Bean
    @ServiceConnection
    RedisContainer redisContainer() { // not
        ↪ "redis...Template/ConnectionFactory": avoid Boot's bean names
        return REDIS;
    }
}
```

- @DataRedisTest yalnızca Redis altyapısını (StringRedisTemplate, connection factory) yükler. Test ettiğiniz bileşenleri @Import ile ekleyin.
- Önbellek testleri cache manager'a ihtiyaç duyar, bu yüzden @SpringBootTest kullanır.
- Tüm testler tek bir Redis'i paylaşır. Her testi bilinen bir durumdan başlatın: @BeforeEach içinde kullandığınız anahtarları silin veya önbelleği invalidate() edin.

! Uyarı

Test hazırlığında Cache.clear() değil, Cache.invalidate() kullanın. clear() daha sonra çalışabilir. invalidate() ise geri döndüğünde önbelleğin boş olduğunu garanti eder.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Önbellek bir veritabanı değildir. Redis veri kaybedebilir (yeniden başlatma, bellek sınırı ve eviction). Asıl veriyi veritabanında tutun ve önbellek boşken uygulamanın, daha yavaş da olsa, çalıştığından emin olun.

- **Yapın:** Her önbelleğe bir TTL verin. Kaynak değişince kayıtları silin veya güncelleyin.
- **Yapmayın:** Bir @Cacheable metodunu aynı sınıftan çağırıp önbellekten gelmesini beklemeyin. Çağrı proxy'yi atlar.
- **Yapın:** Java serileştirmesi değil JSON saklayın. redis-cli ile okunabilir ve sınıf değişikliklerine daha dayanıklıdır.
- **Yapmayın:** null veya "bulunamadı" sonucunu düşünmeden önbelleğe almayın. Sonradan eklenen bir ürün, kayıt süresi dolana kadar görünmez kalır.
- **Yapın:** "Oku, değiştir, yaz" yerine atomik komutlar (INCR, ZINCRBY) kullanın.

- **Yapmayın:** Üretimde KEYS * kullanmayın. Tüm anahtarları tararken Redis'i bloklar. Bunun yerine SCAN kullanın.

5. Özet

- Önbellek, pahalı ve seyrek değişen verinin kopyalarını tutar. Redis, tüm uygulama örneklerine ortak tek bir önbellek sağlar.
- @Cacheable önbellek üzerinden okur, @CachePut önbelleğe de yazar, @CacheEvict kayıtları siler.
- Bir RedisCacheManagerBuilderCustomizer, önbellek başına TTL, JSON serileştirme ve anında yazmayı ayarlar.
- Sorted set, list ve atomik sayaçlar; sıralama, geçmiş ve sayma problemlerini doğrudan Redis'te çözer.
- Pub/sub mesajları yalnızca o anki abonelere iletir. Spring Session, HTTP oturumunu Redis'te saklar.
- @DataRedisTest ve Testcontainers bunların hepsini gerçek bir Redis'e karşı test eder.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Framework — Cache Abstraction](#)
- [Spring Boot — Caching](#)
- [Spring Data Redis Reference · Redis Cache · Pub/Sub Messaging](#)
- [Spring Session — Redis](#)
- [Redis — Data Types](#)